

Data Version Control (DVC)

Managing Datasets & ML Models

MLOps Bootcamp

Instructor: Asadullah

Introduction & Context

- Previously, we tracked experiments using **MLflow** locally.
- Next step: Setting up a remote tracking server for **collaborative work**.
- Before that, we need to learn about essential MLOps tools like **DVC** and **DagsHub**.
- DagsHub provides an AI platform for vision, audio, and document data with remote repository features.
- But first, let's master **DVC (Data Version Control)**.

What is DVC?

Data Version Control manages and versions:

Core Capabilities

- Images
- Audio & Video
- Text files & Datasets
- Machine Learning Models

Why use it?

It organizes your ML modeling process into a **repeatable** and **reproducible workflow**.

In the data ingestion phase, data changes constantly. DVC helps track these changes.

Git vs. DVC

Git

Specifically used for versioning **Code**.

Tracks changes in scripts, configuration files, and text.

Cannot handle large datasets or model files effectively.

DVC

Specifically used for versioning **Data & ML Models**.

Tracks large files, datasets, and model artifacts.

Works **on top of Git** — they are used together.

Initial Setup

Start by creating a fresh environment and initializing Git:

```
conda create -p venv python=3.9 -y
conda activate venv/

git init
```

Install DVC in your environment:

```
pip install dvc

dvc init
```

**Note: DVC init creates a .dvc folder with config and cache directories. Commit these initial files to git.*

Tracking Data with DVC

Let's create a dataset file and track it:

```
mkdir data  
echo "This is the first version" > data/data.txt
```

Now, tell DVC to track this file (similar to git add):

```
dvc add data/data.txt
```

What happens?

- DVC creates data.txt.dvc (a metadata file containing an MD5 hash).
- DVC adds data.txt to .gitignore (Git will no longer track the raw data).

How DVC Works: Hashing & Cache

1. The .dvc File

When you run `dvc add`, DVC reads your data and creates a unique **MD5 hash**.

This hash acts as a pointer/reference to the actual data.

2. The Cache

The actual data is moved to a hidden local cache folder: `.dvc/cache`.

The file in the cache is named using the hash.

Git's Role: Git tracks the `.dvc` file and `.gitignore`, NOT the raw data. The `.dvc` file tells the system which version of the data to pull from the cache.

```
git add data/data.txt.dvc data/.gitignore
git commit -m "Track data v1 with DVC"
```

Updating Data (Versioning)

Let's modify our dataset to create version 2:

```
echo "This is the second version" > data/data.txt
```

Track the changes with DVC and Git:

```
dvc add data/data.txt  
git add data/data.txt.dvc  
git commit -m "Data updated to v2"
```

What changed?

- DVC generates a **new MD5 hash** because the content changed.
- The new data is saved as a new file in the .dvc/cache.
- The old version remains safely stored in the cache!

Switching Between Data Versions

How do we revert to version 1 while keeping our code at version 1 as well?

Step 1: Use Git to checkout the specific commit

```
git log --oneline # Find the commit hash for v1
git checkout <commit_hash>
```

Notice: Git updates the .dvc file to the old hash, but the data.txt file in your folder is still version 2!

Step 2: Use DVC to update the actual data

```
dvc checkout
```

Now data.txt is restored to version 1. **Git and DVC must be used together!**

Returning to Latest & Remote Storage

To go back to your most recent version:

```
git checkout master  
dvc checkout
```

Remote Storage (Future Topic)

Currently, our cache is local. What if models are larger than 500MB?

- Git cannot handle huge files.
- DVC allows you to push your cache to **Remote Repositories**.

Supported Remote Storage options:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage
- Google Drive / DagsHub

Summary

Key Takeaways

- **Git** is for code, **DVC** is for data/models.
- DVC uses **MD5 hashing** to track changes in large files.
- Raw data is stored in the **.dvc/cache** and ignored by Git.
- Git tracks the **.dvc** pointer files.
- Use git checkout + dvc checkout to switch data versions.

Thank You for Watching!

[Like & Subscribe to Asadullah Al](#)

DAGsHub & DVC

Remote Data Versioning & Collaboration

MLOps Bootcamp

Instructor: Asadullah

What is DAGsHub?

DAGsHub is an AI platform for vision, audio, and document data. It provides a powerful **remote repository** for data science and machine learning projects.

Core Capabilities

- Experiment Tracking (MLflow)
- Model Registry
- Data & Model Lineage
- Dataset Management

Why use it?

It allows teams to work **collaboratively**. Multiple developers can view, compare, and select the best models.

Easily integrates with TensorFlow, scikit-learn, AWS, GitHub, GitLab, etc.

GitHub vs. DAGsHub

GitHub

Specifically used for versioning **Code**.

Tracks scripts, configuration files, and text.

Lacks built-in features for large datasets or experiment tracking.

DAGsHub

Tracks **Code** (via Git)

Tracks **Data & Models** (via DVC) - provides 10GB S3 storage!

Tracks **Experiments** (via MLflow)

Provides an end-to-end MLOps stack.

Getting Started

Create a free account on DAGsHub using your GitHub or Google account. Create a **Blank Repository** (e.g., "demo-dagshub").

Clone the empty repository to your local machine and set up the environment:

```
git clone https://dagshub.com/<username>/demo-dagshub.git
cd demo-dagshub

conda create -p venv python=3.10 -y
conda activate venv/
```

Create a .gitignore file and add venv/ to it so your environment isn't tracked by Git.

Requirements & First Commit

Create a requirements.txt file with the necessary libraries:

```
dagsHub  
dvc  
dvc[s3]
```

Install dependencies and make your first commit to track the code:

```
pip install -r requirements.txt  
  
git add README.md  
git commit -m "first commit"  
git branch -M main  
git push -u origin main
```

Tracking Data with DVC

Create a data/ folder and add a dataset (e.g., data.csv). Now, initialize DVC and track the data:

```
dvc init  
dvc add data/data.csv
```

What happens behind the scenes?

- DVC generates a data.csv.dvc file (contains an MD5 hash pointer).
- DVC adds data.csv to .gitignore (Git ignores the raw data).

Track the DVC pointer files with Git:

```
git add data/.gitignore data/data.csv.dvc  
git commit -m "added data.csv with DVC"
```

Configuring Remote Storage

DAGsHub provides an S3 bucket for your data. Go to your DAGsHub repo -> **Remote** -> **Data** to find your credentials. Set up the DVC remote in your terminal:

```
dvc remote add origin s3://dagshub/<repo>.dvc
dvc remote modify origin endpointurl https://dagshub.com/<repo>.s3

dvc remote modify origin --local access_key_id <your_key>
dvc remote modify origin --local secret_access_key <your_secret>
```

**Note: You need the dvc[s3] library installed for this to work, as it uses Boto3 internally.*

Pushing to DAGsHub

Now that the remote is configured, push your data to the DAGsHub S3 bucket, and push your code to the Git repository:

```
dvc push -r origin  
git push origin main
```

What happens on DAGsHub?

- Your code is available in the repository files.
- Your dataset appears as a **DVC managed file** (blue color).
- An automatic **Data Pipeline** visualization is created.

Updating Data (Versioning)

Let's modify our dataset (e.g., remove some rows from data.csv). To track this new version:

```
dvc add data/data.csv
```

The data.csv.dvc file updates with a new MD5 hash. Commit and push the changes:

```
git add .  
git commit -m "second commit - updated data"  
  
dvc push -r origin  
git push origin main
```

On DAGsHub, you can click on the file **History** to visually compare the differences between version 1 and version 2 of your dataset!

Summary

Key Takeaways

- **DAGsHub** centralizes your code, data, and experiments.
- **Git** handles the code, while **DVC** handles the heavy data/models.
- DVC uses **MD5 hashing** and remote S3 storage to version data seamlessly.
- You can easily compare different versions of datasets directly on the DAGsHub UI.

Thank You for Watching!

[Like & Subscribe to Asadullah AI](#)